PRACTICAL - 02

Aim: Smart thermostat (Thermostat.java + Driver.java)

Code Screenshot:

Thermostat.java

```java
public class Thermostat {
    private String location;
    private int temperature;
    private static final int MIN = 16;
    private static final int MAX = 30;
    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;
        this.temperature = (startTemp >= MIN && startTemp <= MAX) ? startTemp : 22;
        activeCount++;
    }

    public Thermostat(String location) {
        this(location, 22);
    }

    public void raise() {
        if (temperature < MAX) temperature++;
        else System.out.println("Already at maximum (30)");
    }

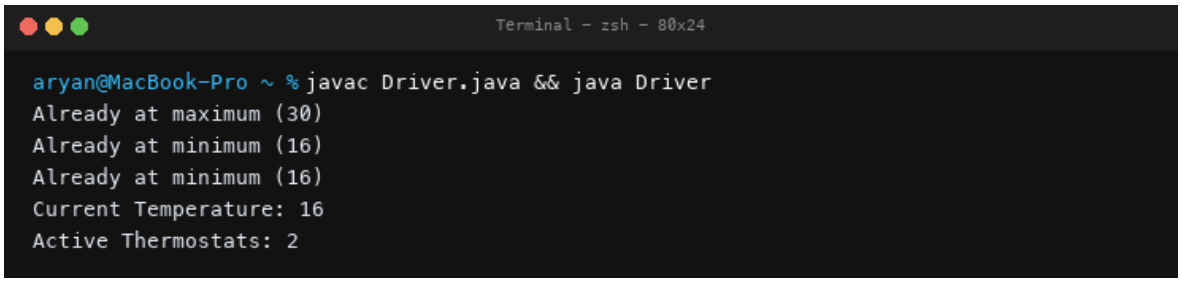
    public void lower() {
        if (temperature > MIN) temperature--;
        else System.out.println("Already at minimum (16)");
    }

    public int getTemperature() { return temperature; }
    public static int getActiveCount() { return activeCount; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Thermostat t1 = new Thermostat("Living Room", 25);
        Thermostat t2 = new Thermostat("Bedroom");
        for (int i = 0; i < 10; i++) t1.raise();
        for (int i = 0; i < 20; i++) t1.lower();
        System.out.println("Current Temperature: " + t1.getTemperature());
        System.out.println("Active Thermostats: " + Thermostat.getActiveCount());
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned encapsulation principles by restricting direct field access with private.
- Implemented constructor overloading and constructor chaining using this(...).
- Managed shared class state across instances using static variables.
- Enforced boundary constraints within mutator methods.
- Understood how objects maintain independent internal state.

Aim: Cinema show (CinemaShow.java + Driver.java)

Code Screenshot:

CinemaShow.java

```java
public class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        this.seatsAvailable = capacity;
    }

    public CinemaShow(String title) {
        this(title, 100);
    }

    public boolean book(int n) {
        if (n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

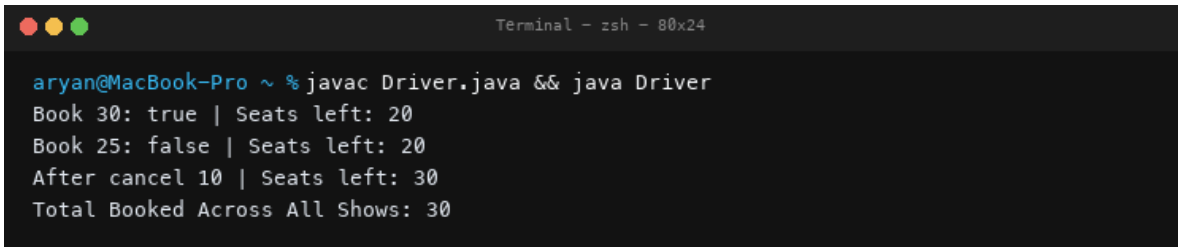
    public void cancel(int n) {
        seatsAvailable = Math.min(capacity, seatsAvailable + n);
    }

    public int getSeatsAvailable() { return seatsAvailable; }
    public static int getTotalBooked() { return totalBooked; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        CinemaShow show = new CinemaShow("Inception", 50);
        System.out.println("Book 30: " + show.book(30) + " | Seats left: " + show.getSeatsAvailable());
        System.out.println("Book 25: " + show.book(25) + " | Seats left: " + show.getSeatsAvailable());
        show.cancel(10);
        System.out.println("After cancel 10 | Seats left:" + show.getSeatsAvailable());
        System.out.println("Total Booked Across All Shows:" + CinemaShow.getTotalBooked());
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modeling real-world transactional entities in Java.
- Implemented defensive programming logic to protect capacity bounds.
- Tracked global operational metrics with static member variables.
- Applied boolean response status for state-modifying actions.
- Practiced clean method signatures and separation of concerns.